



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

实验报告

实验三：参数估计与非参数估计

姓名：孙沐赞

学号：2311534

专业：计算机科学与技术

2025 年 11 月 8 日

目录

一、 实验目的 3

二、 实验环境 3

三、 数据集设置 3

 (一) 基本概念介绍 3

 1. 均值向量与协方差矩阵 3

 2. 先验概率 3

 (二) 分布参数 3

 (三) 先验概率设置 4

 (四) 数据集生成 4

 (五) 数据集图形化展示 5

四、 初级实验要求及实现 6

 (一) 初级实验要求：多类别参数估计与分类性能分析 6

 (二) 基本概念介绍 6

 1. 参数估计 6

 2. 非参数估计 6

 3. 似然率测试规则 7

 4. 最大后验概率规则 7

 (三) 初级实验代码实现 7

 1. 分类器初始化 7

 2. 决策规则实现 7

 3. 评估工具实现 9

 (四) 初级实验结果可视化 9

 (五) 初级实验结果分析 10

 (六) 初级实验总结 11

五、 中级实验要求及实现 11

 (一) 中级实验要求：核密度估计的多参数优化研究 11

 (二) 基本概念介绍 11

 1. 高斯核函数估计 11

 2. 带宽参数 12

 3. 概率密度 12

 (三) 中级实验代码实现 12

 1. 高斯核密度估计 12

 2. KDE 分类器实现 13

 3. 5 折交叉验证实现 14

 (四) 中级实验结果可视化 15

 1. 中级实验结果分析 17

 (五) 中级实验总结 18

六、 高级实验要求及实现	18
(一) 高级实验要求: k-近邻密度估计的全面比较分析	18
(二) 高级实验代码实现	18
1. 分类器初始化与训练	18
2. k-NN 概率密度估计	19
3. 实验执行	20
(三) 高级实验结果可视化	21
1. 高级实验结果分析	22
(四) 高级实验总结	22
七、 实验总结与讨论	23
(一) 实验中每种方法最好性能对比可视化	23
(二) 主要发现	23
(三) 方法优缺点讨论	24
(四) 心得体会	24

一、实验目的

1. 掌握参数估计方法在模式分类中的应用
2. 理解非参数估计的基本原理和实现方法
3. 比较参数方法和非参数方法在不同数据分布下的性能
4. 分析先验概率对分类器性能的影响

二、实验环境

- Python 3.8+
- 主要依赖库: numpy, matplotlib, scipy, scikit-learn, seaborn, pandas
- 开发环境: Jupyter Notebook / PyCharm / VS Code

三、数据集设置

(一) 基本概念介绍

1. 均值向量与协方差矩阵

均值向量 (μ) 与协方差矩阵 (Σ) 是描述多维数据 (如本实验中的二维高斯分布) 特征的核心参数。均值向量 μ 是一个 d 维向量 (本实验中 $d = 2$)，它的各个分量分别表示数据在相应维度上的期望值或均值。在几何上，均值向量定义了数据簇的中心位置。

协方差矩阵 Σ 是一个 $d \times d$ 的对称矩阵，它描述了数据在不同维度上的离散程度 (方差) 以及维度间的线性相关性 (协方差)。矩阵的对角线元素 Σ_{ii} 表示第 i 维特征的方差，非对角线元素 Σ_{ij} 表示第 i 维与第 j 维特征之间的协方差。协方差矩阵决定了数据簇的形状、大小和朝向。

2. 先验概率

先验概率 ($P(w_i)$) 是指在观测到任何具体的数据样本 x 之前，我们对某个类别 w_i 出现的可能性的初始信念或背景知识。它反映了各个类别在总体数据中固有的分布比例。我们举一个例子来更好地描述什么是先验概率，在本实验的数据集 B 中，类别 w_1 的先验概率 $P(w_1) = 0.5$ ，意味着在生成数据时，有 50% 的样本被设定为来自类别 w_1 。

(二) 分布参数

本实验生成三种数据集，每种包含四个分布模型：

均值向量：

$$\begin{aligned}\mu_1 &= [1, 2], & \mu_2 &= [3, 5], \\ \mu_3 &= [6, 3], & \mu_4 &= [4, 7]\end{aligned}$$

协方差矩阵：

$$\Sigma_1 = \Sigma_2 = \Sigma_3 = \Sigma_4 = 1.5\mathbf{I}$$

(三) 先验概率设置

表 1: 三种数据集的先验概率设置

数据集	$p(w_1)$	$p(w_2)$	$p(w_3)$	$p(w_4)$	样本数量
A (均衡先验)	0.25	0.25	0.25	0.25	1200
B (偏斜先验 1)	0.5	0.2	0.2	0.1	1200
C (偏斜先验 2)	0.1	0.1	0.3	0.5	1200

(四) 数据集生成

在本次实验中，我们需要按照上述参数设置手动完成数据集的生成。

首先，定义包含了四个类别分布参数（均值与协方差）以及三个数据集先验概率与样本总数的配置字典 `datasets_config`，具体代码实现如下：

Listing 1: 数据集参数与先验概率配置

```

1  # —— 1. 数据集参数设置 ——
2  means = {
3      1: np.array([1, 2]), 2: np.array([3, 5]),
4      3: np.array([6, 3]), 4: np.array([4, 7])
5  }
6  covs = {
7      1: np.array([[1.5, 0], [0, 1.5]]), 2: np.array([[1.5, 0], [0,
8          1.5]]),
9      3: np.array([[1.5, 0], [0, 1.5]]), 4: np.array([[1.5, 0], [0,
10         1.5]])
11 }
12
13 # —— 先验概率设置 (表1) ——
14 datasets_config = {
15     'A': {'priors': [0.25, 0.25, 0.25, 0.25], 'N': 1200, 'title':
16         'Dataset_A (均衡先验)'},
17     'B': {'priors': [0.5, 0.2, 0.2, 0.1], 'N': 1200, 'title': '
18         Dataset_B (偏斜先验1)'},
19     'C': {'priors': [0.1, 0.1, 0.3, 0.5], 'N': 1200, 'title': '
20         Dataset_C (偏斜先验2)'}
21 }
```

数据集的生成通过一个自定义函数 `generate_dataset` 实现。该函数遍历指定配置中的先验概率，依据样本总数 `N` 计算该类别应生成的样本数 `n_samples`。

我们通过调用 `numpy.random.multivariate_normal` 函数，从具有指定均值和协方差的多元高斯分布中抽取相应数量的样本。

具体代码如下：

Listing 2: 数据集生成核心函数

```

1  def generate_dataset(config):
2      """根据配置生成数据集"""
```

```

3     X = []
4     y = []
5     for i, prior in enumerate(config['priors']):
6         n_samples = int(config['N'] * prior)
7         class_label = i + 1
8         # 核心：从多元高斯分布中抽取样本
9         X.append(np.random.multivariate_normal(means[class_label],
10        covs[class_label],
11        n_samples))
12        y.append(np.full(n_samples, class_label))
13
14    X = np.concatenate(X)
15    y = np.concatenate(y)
16
17    # 打乱数据，确保样本的随机性
18    indices = np.arange(X.shape[0])
19    np.random.shuffle(indices)
20    X = X[indices]
21    y = y[indices]
22
23    return X, y

```

最后，通过循环遍历配置字典来执行生成函数，从而构建出实验所需的 A、B、C 三个数据集，并将其存储于一个字典 `datasets` 中以供后续调用，代码实现如下：

Listing 3: 执行数据集生成

```

1     # —— 4. 生成数据 ——
2     datasets = {}
3     for name, config in datasets_config.items():
4         X, y = generate_dataset(config)
5         datasets[name] = {'X': X, 'y': y, 'config': config}
6         print(f"Generated {name} with {len(y)} samples.")

```

(五) 数据集图形化展示

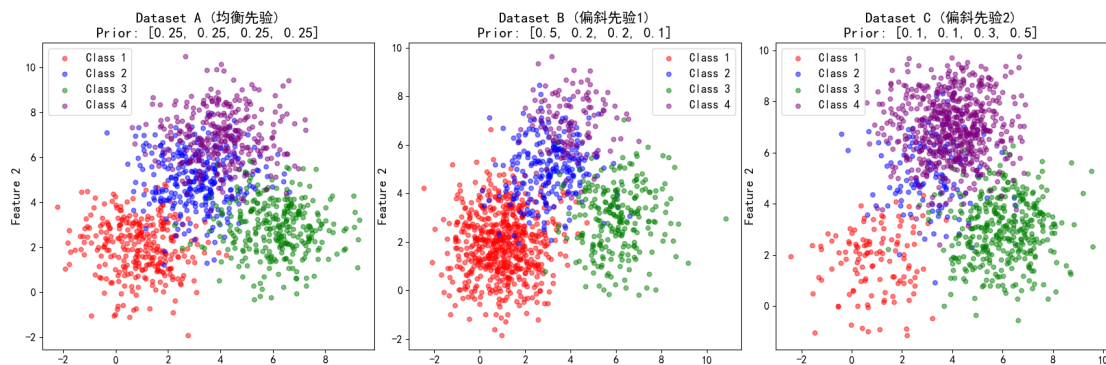


图 1: 数据集样本分布散点图

三个数据集的样本分布散点图从左至右依次为：数据集 A（均衡先验）、数据集 B（偏斜先验 1）与数据集 C（偏斜先验 2）。图1中，四种不同颜色（红色、蓝色、绿色、紫色）分别对应类别 1 至类别 4。可以清晰地观察到，所有类簇均围绕其设定的均值向量聚集，且由于协方差矩阵相同，各个类簇呈现出相似的圆形分布形态。各数据集中类簇的样本数量直观地反映了表 1 所设定的先验概率：数据集 A 的四类样本数量大致均衡；数据集 B 中类别 1（红色）的样本量显著多于其他类别；而数据集 C 中类别 4（紫色）的样本量占据主导地位。此外，各类别之间存在明显的重叠区域，这对分类器的性能构成了挑战。

四、 初级实验要求及实现

（一） 初级实验要求：多类别参数估计与分类性能分析

1. 按照指定参数设置生成三个数据集 A、B、C
2. 在每个数据集上分别实现和应用：
 - 似然率测试规则分类器（四类别）
 - 最大后验概率规则分类器（四类别）
3. 计算并比较：
 - 总体分类错误率
 - 每个类别的分类准确率
 - 混淆矩阵分析
4. 绘制：
 - 三个数据集的样本分布散点图（四类别用不同颜色）

（二） 基本概念介绍

1. 参数估计

参数估计是一种基于先验知识的概率密度估计方法。该方法假设样本数据服从某一特定的、已知形式的概率分布模型（如高斯分布），而模型的具体参数（如均值 μ 和协方差 Σ ）是未知的。参数估计的目标就是利用训练样本来估计这些未知参数，从而得到类条件概率密度函数 $p(x|w_i)$ 的完整数学表达。

2. 非参数估计

非参数估计则不预先假设概率密度的具体分布形式，而是直接从训练样本本身出发来估计未知的概率密度，其估计结果完全依赖于样本数据。本实验中的核密度估计（KDE）与 k-近邻（k-NN）密度估计均属于此类方法，它们通过在待测点 x 附近的一个局部区域内考察训练样本的分布情况来估计其概率密度。

3. 似然率测试规则

似然率测试规则是一种分类决策规则。它基于最大似然准则，即在进行决策时，仅考虑样本 x 相对于各个类别的类条件概率（似然） $p(x|w_i)$ 。分类器将样本 x 判给能使似然 $p(x|w_i)$ 最大的那个类别 w_i 。此规则隐含地假设了所有类别的先验概率 $P(w_i)$ 是均等的，因此在先验概率不均衡时可能并非最优决策。

4. 最大后验概率规则

最大后验概率规则是一种基于贝叶斯定理的决策规则。它的目标是最大化后验概率 $P(w_i|x)$ ，即“给定观测样本 x ，它属于类别 w_i 的概率”。根据贝叶斯公式， $P(w_i|x) \propto p(x|w_i)P(w_i)$ 。因此，MAP 规则将样本 x 判给能使似然 $p(x|w_i)$ 与先验概率 $P(w_i)$ 的乘积最大的类别 w_i 。与 LRT 不同，MAP 规则显式地融合了关于类别分布的先验知识，在先验概率不均衡时通常能提供更优的分类性能。

（三） 初级实验代码实现

初级要求的核心是实现参数估计分类器。由于本实验的前提是已知数据生成所用的高斯分布参数（均值与协方差），我们构建了一个 `ParametricClassifier` 类，该类在初始化时便利用这些真实参数。

1. 分类器初始化

分类器在其实例化（`__init__`）时，接收真实的均值与协方差作为参数。其关键步骤是利用 `scipy.stats.multivariate_normal` 库函数，为四个类别中的每一个预先创建一个概率密度函数（PDF）对象（存储于 `self.pdfs_`）。这些对象封装了高斯分布的数学模型，能用于后续高效计算任意数据点 x 的似然 $p(x|w_i)$ 。

Listing 4: 参数估计分类器初始化

```

1 class ParametricClassifier:
2     """
3     参数估计分类器。
4     我们“已知”真实的均值（means）和协方差（covs）。
5     """
6     def __init__(self, means, covs):
7         self.means = means
8         self.covs = covs
9         self.classes_ = list(means.keys()) # 类别 [1, 2, 3, 4]
10
11         # 预先创建 PDF 对象，用于计算 P(x|w_i)
12         # 这是参数估计的核心：我们用已知的均值和协方差来构建模型
13         self.pdfs_ = {c: multivariate_normal(mean=means[c], cov=covs[c])
14                        for c in self.classes_}

```

2. 决策规则实现

分类器的核心逻辑集中于 `predict_log_proba` 与 `predict` 方法。为了数值稳定性（避免极小概率值连乘导致的浮点数下溢），所有计算均在对数空间中进行。

`predict_log_proba` 方法实现了两种决策规则的统一计算。它首先通过调用 `self.pdfs_[c].logpdf(X)` 来获取所有样本点 X 对应于类别 c 的对数似然 $\log p(x|w_i)$ 。

随后，代码根据传入的 `rule_type` 参数进行分支：

- **似然率测试 (LRT)**: 直接使用对数似然 $\log p(x|w_i)$ 作为决策依据。
- **最大后验概率 (MAP)**: 在对数似然的基础上，加上对应类别的对数先验概率 $\log P(w_i)$ ，从而计算对数后验概率 $\log p(x|w_i) + \log P(w_i)$ 。

`predict` 方法则调用 `predict_log_proba` 来获取决策分数矩阵，并使用 `np.argmax` 沿类别维度 (`axis=1`) 找出每个样本得分最高的类别索引，从而完成分类决策。

Listing 5: LRT 与 MAP 决策规则实现

```

1 def predict_log_proba(self, X, priors, rule_type='MAP'):
2     """
3     计算对数后验概率或对数似然。
4     """
5     log_probs = []
6     for c in self.classes_:
7         # P(x|wi)
8         # 使用 logpdf 来防止数值下溢，速度也更快
9         log_likelihood = self.pdfs_[c].logpdf(X)
10
11        if rule_type == 'MAP':
12            # MAP 规则: log(P(x|wi)) + log(P(wi))
13            prior_prob = priors[c-1] # 索引 0-3 对应类别 1-4
14
15            if prior_prob == 0:
16                log_prior = -np.inf
17            else:
18                log_prior = np.log(prior_prob)
19
20            log_posterior = log_likelihood + log_prior
21            log_probs.append(log_posterior)
22        else: # LRT
23            # 似然率测试: log(P(x|wi))
24            log_probs.append(log_likelihood)
25
26        return np.stack(log_probs, axis=1)
27
28 def predict(self, X, priors, rule_type='MAP'):
29     """进行分类"""
30     log_posteriors = self.predict_log_proba(X, priors, rule_type)
31
32     # 找到最大概率的索引 (0-3)，然后映射回类别 (1-4)
33     pred_indices = np.argmax(log_posteriors, axis=1)
34     return np.array(self.classes_)[pred_indices]
```

3. 评估工具实现

我们手动实现了准确率和混淆矩阵的计算函数，以避免对 `scikit-learn` 库的依赖。

Listing 6: 手动实现评估函数

```

1 def manual_accuracy(y_true, y_pred):
2     """手动计算准确率"""
3     return np.mean(y_true == y_pred)
4
5 def manual_confusion_matrix(y_true, y_pred, n_classes=4):
6     """手动计算混淆矩阵"""
7     cm = np.zeros((n_classes, n_classes), dtype=int)
8     for i in range(len(y_true)):
9         true_label_idx = int(y_true[i] - 1) # 类别 1-4 对应索引 0-3
10        pred_label_idx = int(y_pred[i] - 1)
11        cm[true_label_idx, pred_label_idx] += 1
12    return cm

```

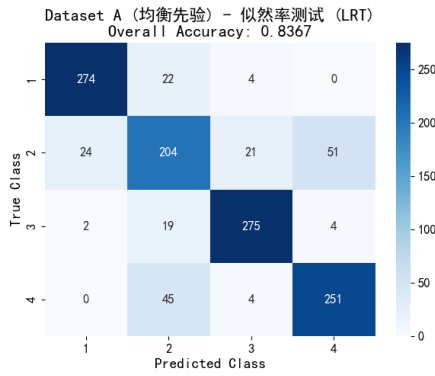
(四) 初级实验结果可视化

表 2 详细记录了参数估计方法在三个数据集上 (A, B, C) 分别应用两种决策规则 (LRT, MAP) 时的总体分类准确率与各类别准确率 (召回率)。

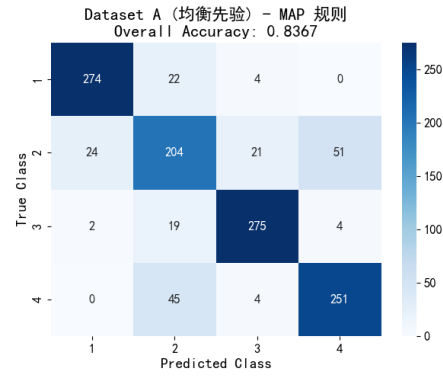
表 2: 参数估计方法在不同数据集上的分类准确率

数据集	分类规则	总体准确率	类别 1 准确率	类别 2 准确率	类别 3 准确率	类别 4 准确率
A	似然率测试	0.8367	0.9133	0.6800	0.9167	0.8367
	MAP 规则	0.8367	0.9133	0.6800	0.9167	0.8367
B	似然率测试	0.8725	0.9317	0.7333	0.9125	0.7750
	MAP 规则	0.8808	0.9650	0.7458	0.9125	0.6667
C	似然率测试	0.8400	0.9333	0.7083	0.8972	0.8133
	MAP 规则	0.8783	0.9167	0.3583	0.9222	0.9483

图 2、3 与 4 分别展示了与表 2 对应的分类混淆矩阵热图。

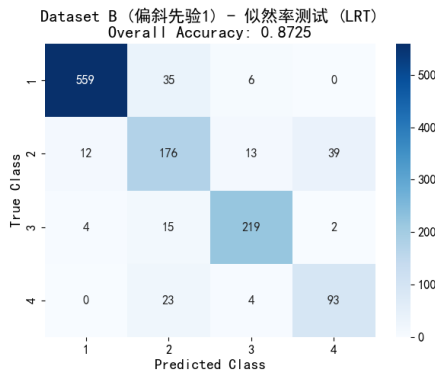


(a) 似然率测试 (LRT) 规则

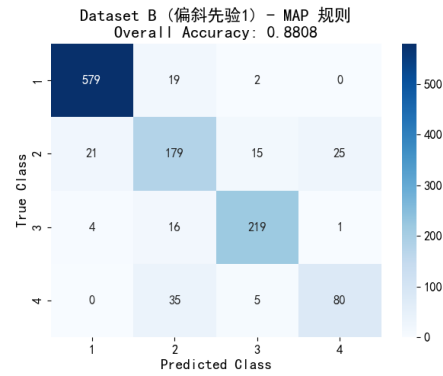


(b) 最大后验概率 (MAP) 规则

图 2: 数据集 A (均衡先验) 的参数估计分类结果

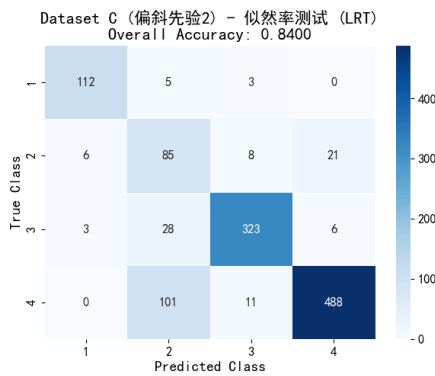


(a) 似然率测试 (LRT) 规则

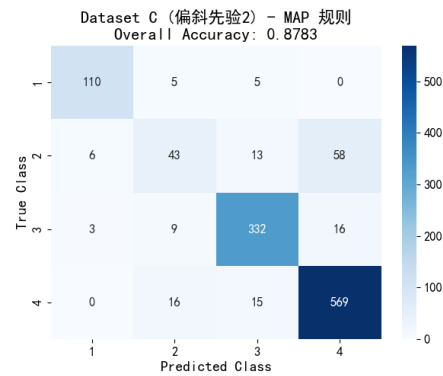


(b) 最大后验概率 (MAP) 规则

图 3: 数据集 B (偏斜先验 1) 的参数估计分类结果



(a) 似然率测试 (LRT) 规则



(b) 最大后验概率 (MAP) 规则

图 4: 数据集 C (偏斜先验 2) 的参数估计分类结果

(五) 初级实验结果分析

在数据集 A (均衡先验) 中, LRT 与 MAP 规则的表现完全一致, 总体准确率均为 0.8367。这符合贝叶斯决策理论: 当先验概率均等时, MAP 决策等价于 LRT 决策。

在数据集 B（偏斜先验 1）中，类别 1 的先验概率 $P(w_1) = 0.5$ 占主导。MAP 规则的总体准确率 (0.8808) 高于 LRT (0.8725)。深入分析表 2 和图 3 可见，MAP 规则通过利用高先验知识，显著提升了类别 1 的准确率 (0.9650 vs 0.9317)，尽管这牺牲了低先验类别 4 的准确率 (0.6667 vs 0.7750)，但最终获得了更高的总体性能。

在数据集 C（偏斜先验 2）中，先验概率的影响更为显著。MAP 规则的总体准确率 (0.8783) 远高于 LRT (0.8400)。图 4 显示，LRT 将大量样本误判为类别 2（如将 101 个真实类别 4 判为类别 2）。而 MAP 规则通过引入 $P(w_4) = 0.5$ 的强先验，极大地修正了这一偏差，使类别 4 的准确率从 0.8133 飙升至 0.9483。其代价是低先验类别 2 的准确率从 0.7083 骤降至 0.3583。

综上所述，参数估计的实验结果清晰地揭示了先验概率对分类器性能的决定性影响。在先验信息已知且不平衡时，MAP 规则通过牺牲低先验类别的性能来换取高先验类别的性能提升，从而在总体准确率上实现了优于 LRT 的决策。

（六） 初级实验总结

在数据集 A（均衡先验）中，LRT 与 MAP 规则的表现完全一致，这符合贝叶斯决策理论，即当先验概率均等时，MAP 决策退化为 LRT 决策。

然而，在数据集 B 和 C（偏斜先验）中，MAP 规则的总体分类准确率均明显优于 LRT 规则。通过分析混淆矩阵可以发现，LRT 规则由于忽略了先验信息，在处理类别分布不均的数据时表现不佳，易将样本误判为似然稍高但先验极低的类别。MAP 规则通过将先验概率 $P(w_i)$ 纳入决策考量，有效地修正了这一偏差，它倾向于将决策边界向低先验概率的类别推移，从而在高先验概率的类别上获得了更高的准确率，并最终提升了总体分类性能。

在初级实验中，我们成功验证了参数估计方法的基本应用，并从数据上有力地证明了在先验概率已知且不平衡时，MAP 规则是比 LRT 更优的贝叶斯决策策略。

至此，初级实验完成！

五、 中级实验要求及实现

（一） 中级实验要求：核密度估计的多参数优化研究

1. 使用高斯核函数估计进行四类别的概率密度估计
2. 带宽参数优化：
 - 测试范围： $h \in [0.1, 0.3, 0.5, 0.8, 1.0, 1.5, 2.0]$
 - 使用 5 折交叉验证寻找最优 h 值
3. 在两个分类规则下分别进行：
 - 不同 h 值下的分类准确率比较
 - 最优 h 值的稳定性分析
4. 可视化不同 h 值下的概率密度估计效果

（二） 基本概念介绍

1. 高斯核函数估计

核函数估计是一种核心的非参数概率密度估计技术。其基本思想是在待估计点 x 处，将一个核函数 $K(u)$ 作用于 x 与每个训练样本 x_i 的差值上，并将所有核函数的响应进行叠加和归一

化，以此来估计该点的概率密度。

高斯核函数估计是核函数估计的一种具体实现，它选用高斯函数（正态分布的概率密度函数）作为核函数。其表达式通常为：

$$K(u) = \frac{1}{(\sqrt{2\pi})^d} e^{-\frac{1}{2}u^T u}$$

其中 $u = (x - x_i)/h$ ， h 为带宽参数， d 为数据维度。使用高斯核函数进行密度估计 $p(x)$ 的最终公式为：

$$p(x) = \frac{1}{Nh^d} \sum_{i=1}^N \frac{1}{(\sqrt{2\pi})^d} e^{-\frac{1}{2}(\frac{x-x_i}{h})^T (\frac{x-x_i}{h})}$$

该方法通过在每个样本点上叠加一个“平滑”的高斯“凸起”来拟合整体的数据分布。

2. 带宽参数

带宽参数（ h ）是核密度估计与 k -近邻估计中至关重要的平滑参数。在核密度估计中， h 控制了每个核函数（如高斯核）的“宽度”或影响范围。

- **较小的 h 值：**会导致核函数非常“尖锐”，仅在样本点极近的邻域内产生响应。这使得估计出的密度函数波动剧烈，能够捕捉到数据的精细结构，但也极易受到样本噪声的干扰，导致“过拟合”。
- **较大的 h 值：**会导致核函数非常“扁平”，每个样本点的影响范围都很大。这使得估计出的密度函数过度平滑，掩盖了数据中潜在的结构和峰谷变化，导致“欠拟合”。

因此，选择一个最优的 h 值，以在估计的“偏差”与“方差”之间达成平衡，是核密度估计的核心挑战之一。

3. 概率密度

概率密度是用于描述连续型随机变量分布的函数，通常记为 $p(x)$ 。它并非直接表示某个点 x 发生的概率（连续变量取任意单点的概率恒为零），而是表示 x 落在其一个极小邻域 Δx 内的概率 $P(x \in \Delta x)$ 与该邻域“体积” Δx 的比值。

$$p(x) = \lim_{\Delta x \rightarrow 0} \frac{P(x \in \Delta x)}{\Delta x}$$

概率密度函数在整个定义域上的积分恒为 1。在模式分类的贝叶斯框架中，我们核心关注的是类条件概率密度 $p(x|w_i)$ ，它量化了“假设样本来自类别 w_i ，其特征为 x ”的相对可能性（即似然）。无论是参数估计还是非参数估计，其主要目标都是为了获取这个 $p(x|w_i)$ 。

（三） 中级实验代码实现

中级实验的核心是实现非参数的核密度估计（KDE）分类器，并手动实现 5 折交叉验证来寻找最优带宽参数 h 。

1. 高斯核密度估计

为满足实验要求，我们首先手动实现了高斯核密度估计的对数似然函数 `gaussian_kernel_log_pdf_manual`。此函数不依赖 `scipy` 库，而是基于我们自定义的 `manual_pairwise_sqdist`（计算平方欧氏距离）和 `manual_log_sum_exp`（在对数空间中安全求和）函数。

其计算遵循高斯 KDE 的数学定义：

$$\log p(x) = \log \left(\frac{1}{N} \sum_{i=1}^N K(x, x_i) \right) = \text{logsumexp}(\log K(x, x_i)) - \log N$$

其中 $\log K(x, x_i)$ 是高斯核的对数形式。

具体代码实现如下：

Listing 7: 手动实现高斯核密度估计

```

1 def gaussian_kernel_log_pdf_manual(X_test, X_train, h):
2     """
3     手动计算高斯核密度估计的对数似然 log(P(x))
4     """
5     N, d = X_train.shape
6
7     sq_dists = manual_pairwise_sqdist(X_test, X_train) # (M, N)
8
9     # 计算高斯核 K(u) 的对数
10    log_kernel_vals = - (d/2) * np.log(2 * np.pi * h**2) - (sq_dists / (2
11        * h**2))
12
13    # 求和并取平均
14    log_pdf = manual_log_sum_exp(log_kernel_vals, axis=1) - np.log(N)
15    return log_pdf

```

2. KDE 分类器实现

KDEClassifier 类封装了 KDE 的非参数特性。其 fit 方法仅仅是将训练数据 X 和 y 按类别存储起来，这体现了 KDE “懒惰学习” 的本质。

predict_log_proba 方法与参数估计中的实现逻辑一致：它首先调用 gaussian_kernel_log_pdf_manual 函数计算每个类别的对数似然 $\log p(x|w_i)$ ，然后根据 rule_type 应用 LRT 或 MAP 决策规则，计算最终的决策分数。

具体代码实现如下：

Listing 8: KDE 分类器核心逻辑

```

1 class KDEClassifier:
2     """手动实现KDE分类器"""
3     def __init__(self, h=1.0):
4         self.h = h
5         self.X_train_ = {}
6         self.classes_ = []
7
8     def fit(self, X, y):
9         self.classes_ = np.unique(y)
10        for c in self.classes_:
11            self.X_train_[c] = X[y == c]
12            # ... (省略了样本检查) ...
13    return self

```

```

14
15     def predict_log_proba(self, X_test, priors, rule_type='MAP'):
16         log_probs = []
17         for c in self.classes_:
18             X_train_c = self.X_train_[c]
19
20             if X_train_c.shape[0] == 0:
21                 log_likelihood = np.full(X_test.shape[0], -np
22                     .inf)
23             else:
24                 log_likelihood =
25                     gaussian_kernel_log_pdf_manual(X_test,
26                         X_train_c, self.h)
27
28             if rule_type == 'MAP':
29                 # ... (MAP 规则实现, 同初级实验) ...
30                 log_posterior = log_likelihood + np.log(
31                     priors[int(c)-1])
32                 log_probs.append(log_posterior)
33             else: # LRT
34                 log_probs.append(log_likelihood)
35
36     return np.stack(log_probs, axis=1)
37
38     # ... (predict 方法, 同初级实验, 此处省略) ...

```

3. 5 折交叉验证实现

为寻找最优带宽 h , 我们手动实现了 `run_kde_cv` 函数。该函数首先将数据集索引随机打乱并平均分为 5 折 (`n_splits = 5`)。

在一个外层循环中, 代码遍历所有待测试的 h 值。在内层循环中, 代码执行 5 次训练与验证: 每次取 1 折作为验证集 (`X_val, y_val`), 剩余 4 折作为训练集 (`X_train, y_train`)。模型在训练集上拟合, 并在验证集上评估其 LRT 和 MAP 准确率。

最后, 函数返回一个字典, 其中包含了每个 h 值在 5 折验证中得到的平均准确率。

具体代码实现如下:

Listing 9: 手动实现 5 折交叉验证

```

1  def run_kde_cv(X, y, priors, h_values):
2      """为KDE手动运行5折交叉验证以优化h"""
3      n_splits = 5
4      indices = np.arange(X.shape[0])
5      np.random.shuffle(indices)
6      fold_indices = np.array_split(indices, n_splits)
7
8      scores_map = {h: [] for h in h_values}
9      scores_lrt = {h: [] for h in h_values}
10

```

```

11     for h in h_values:
12         print(f"正在测试带宽 h={h}")
13         for i in range(n_splits):
14             # 1. 划分训练集和验证集
15             val_idx = fold_indices[i]
16             train_idx = np.concatenate([fold_indices[j]
17                                         for j in range(n_splits) if j != i])
18
19             X_train, y_train = X[train_idx], y[train_idx]
20             X_val, y_val = X[val_idx], y[val_idx]
21
22             # 2. 拟合模型
23             kde = KDEClassifier(h=h)
24             kde.fit(X_train, y_train)
25
26             # 3. 评估 (MAP)
27             y_pred_map = kde.predict(X_val, priors, rule_type='
                MAP')
28             scores_map[h].append(manual_accuracy(y_val,
29                                                  y_pred_map))
30
31             # 4. 评估 (LRT)
32             y_pred_lrt = kde.predict(X_val, priors, rule_type='
                LRT')
33             scores_lrt[h].append(manual_accuracy(y_val,
34                                                  y_pred_lrt))
35
36         # 计算平均分
37         avg_scores_map = {h: np.mean(scores_map[h]) for h in h_values}
38         avg_scores_lrt = {h: np.mean(scores_lrt[h]) for h in h_values}
39
40     return avg_scores_map, avg_scores_lrt

```

(四) 中级实验结果可视化

表 3: 不同数据集下的最优带宽参数及对应准确率

数据集	分类规则	最优 h 值	交叉验证准确率	可视化准确率
A	似然率测试	2.0	0.8383	0.8442 ($h=2.0$)
	MAP 规则	2.0	0.8383	
B	似然率测试	0.8	0.8808	0.8917 ($h=0.5$)
	MAP 规则	0.5	0.8883	
C	似然率测试	1.5	0.8483	0.8850 ($h=0.8$)
	MAP 规则	0.8	0.8817	

表 3 汇总了 5 折交叉验证 (CV) 所确定的最优 h 值及其对应的 CV 准确率。最后一列“可视化准确率”记录了图 6至8 中所用 h 值 (括号内) 及其在全数据集上得到的准确率。

KDE 带宽参数 (h) 对准确率的影响 (5折交叉验证)

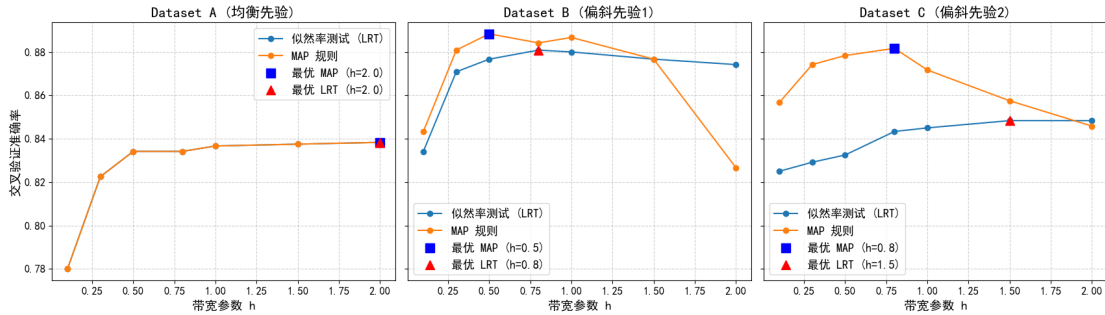


图 5: KDE 带宽参数 h 对分类准确率的影响 (5 折交叉验证)

图 5 直观地展示了带宽参数 h 的选择对 KDE 分类器性能的决定性影响。在所有三个数据集中, 均表现出清晰的“欠拟合-最优-过拟合”模式。当 h 值过小 (如 $h = 0.1$) 时, KDE 模型过拟合训练数据中的噪声, 导致泛化能力差, 准确率低。当 h 值过大 (如 $h = 2.0$) 时, KDE 模型过度平滑, 丢失了类别间的边界细节, 导致欠拟合, 准确率同样下降。最优的 h 值是在模型偏差与方差之间取得平衡的结果。

该图再次验证了先验概率的影响。在数据集 A (均衡先验) 中, LRT 与 MAP 规则的性能曲线几乎重合。而在数据集 B 和 C (偏斜先验) 中, MAP 规则 (橙色线) 的准确率在多数 h 取值下均显著高于 LRT 规则 (蓝色线), 展示了 MAP 规则在处理不均衡数据时的持续优越性。

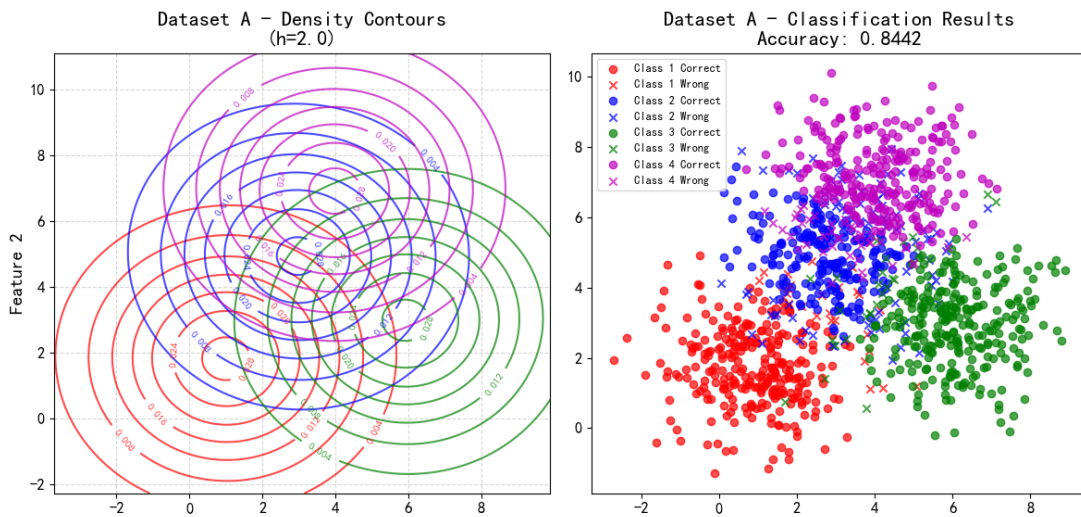


图 6: 数据集 A (均衡先验) 的 KDE 可视化 (使用 $h = 2.0$)

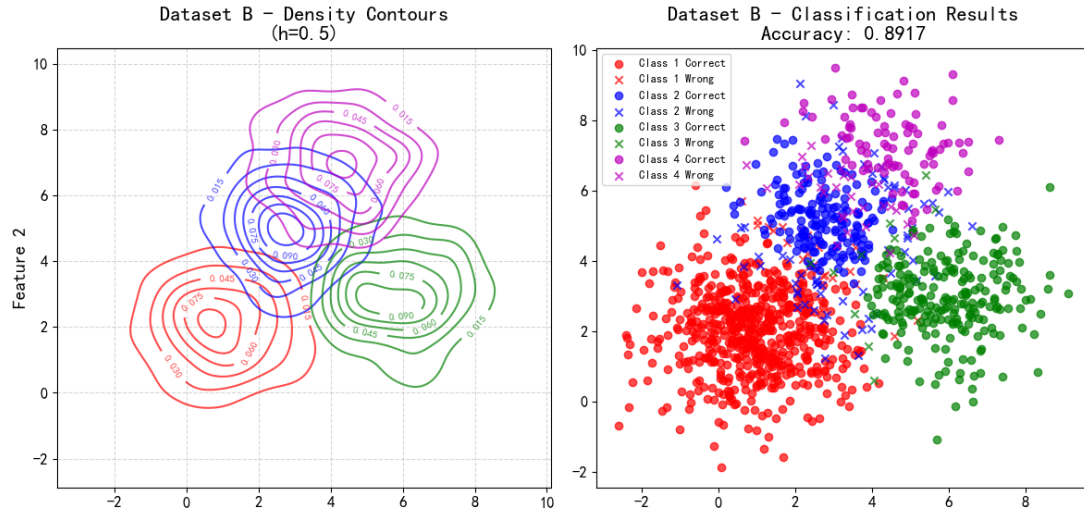
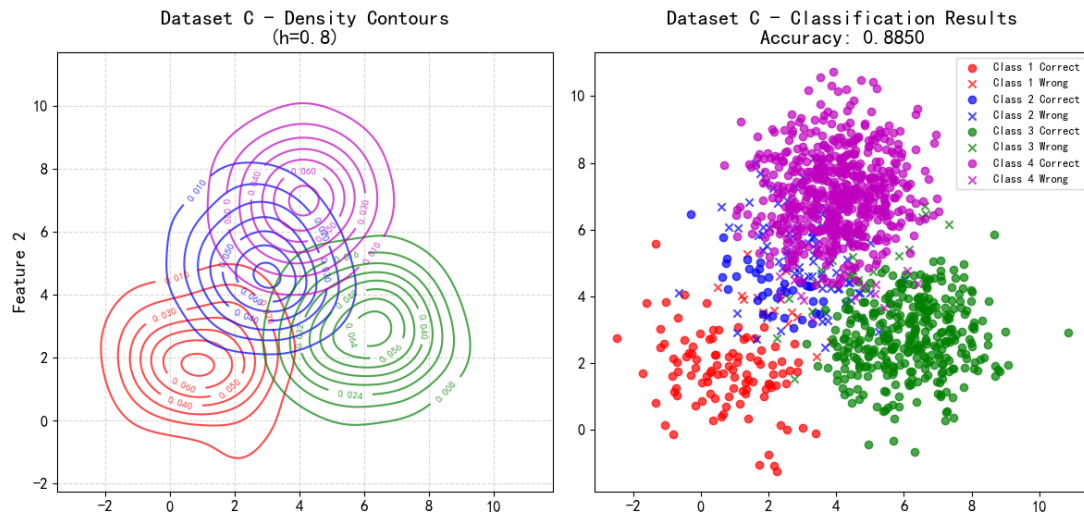
图 7: 数据集 B (偏斜先验 1) 的 KDE 可视化 (使用 $h = 0.5$)图 8: 数据集 C (偏斜先验 2) 的 KDE 可视化 (使用 $h = 0.8$)

图 6、7 与 8 展示了使用特定 h 值的 KDE 可视化结果。每张图的左侧为 KDE 估计出的四个类别的概率密度等高线，右侧为 MAP 规则下的最终分类结果（圆圈'o' 为正确，叉'x' 为错误）。

1. 中级实验结果分析

KDE 作为一种非参数方法，其目标是在不知道数据真实分布（高斯分布）的前提下，从样本中重建该分布。从三张图的左侧可以看出，KDE 成功地从样本数据中拟合出了四个峰值的密度分布，其形状与位置均与真实的数据簇相对应。 h 值对平滑度的控制效果显著：在 $h = 0.5$ （图 7 左）和 $h = 0.8$ （图 8 左）时，等高线相对紧密地包裹了数据簇；而在 $h = 2.0$ （图 6 左）时，等高线则显得极为平滑和宽广，几乎丢失了类簇的独立形态。

三张图的右侧（分类结果图）直观地展示了分类器的性能。绝大多数错误分类（'x' 标记）均

精确地出现在不同类别密度等高线的重叠区域，即类边界处。这符合贝叶斯决策理论，即错误主要发生在后验概率 $P(w_i|x)$ 相近的决策模糊区域。

（五） 中级实验总结

中级实验成功实现了一个基于高斯核的非参数密度估计分类器，我们在中级实验中完成了带宽参数 h 的优化。通过 5 折交叉验证，我们量化了 h 值对分类性能的非线性影响，验证了 h 既不能过小（过拟合）也不能过大（欠拟合）的理论。

在最优 h 值下，KDE 方法在不知道数据真实模型的前提下，取得了接近参数估计方法（初级实验）的分类准确率，证明了其作为一种灵活的非参数估计方法的有效性。同时，实验结果再次确认，无论使用何种密度估计方法（参数或非参数），MAP 决策规则在处理先验概率不均衡的数据集时均表现出显著的优势。

至此，我们完成了中级实验！

六、 高级实验要求及实现

（一） 高级实验要求：k-近邻密度估计的全面比较分析

1. 实现 k-近邻概率密度估计方法
2. 测试 $k = 1, 3, 5, 8, 10$ 五种情况：
 - 计算四个类别的概率密度估计
 - 记录不同 k 值下的分类准确率
3. 深入分析：
 - k 值对密度估计平滑程度的影响
 - 不同类别密度分布特点对 k 值选择的敏感性
4. 可视化展示：
 - 不同 k 值下的概率密度等高线图
 - 分类结果对比图

（二） 高级实验代码实现

高级实验的核心是实现 `KNNDensityClassifier` 类。k-NN 密度估计与 KDE 均属非参数方法，但其原理不同。k-NN 不固定“体积” V （如 KDE 的带宽 h ），而是固定“邻居数” k ，通过寻找包含 k 个邻居所需的体积 $V_i(x)$ 来估计密度。

1. 分类器初始化与训练

与 KDE 相似，k-NN 密度估计也是一种“懒惰学习”方法。其 `fit` 方法的实现非常简洁：它不进行任何复杂的计算，仅仅是将训练数据集 X 和 y 按照其类别 c 划分，并存储在类的成员变量 `self.X_train_` 和 `self.N_i_`（各类样本数）中，以备预测时调用，代码实现如下：

Listing 10: k-NN 密度估计的 fit 方法

```

1 class KNNDensityClassifier:
2     """
3     手动实现 k-NN 密度估计分类器
4     """
5     def __init__(self, k=5):
6         self.k = k
7         self.X_train_ = {} # 按类别存储训练数据
8         self.N_i_ = {}     # 按类别存储样本数
9         self.classes_ = []
10
11    def fit(self, X, y):
12        self.classes_ = np.unique(y)
13        for c in self.classes_:
14            self.X_train_[c] = X[y == c]
15            self.N_i_[c] = self.X_train_[c].shape[0]
16
17        # ... (省略了 k 值健全性检查) ...
18    return self

```

2. k-NN 概率密度估计

k-NN 密度估计的实现完全集中于 `predict_log_proba` 方法。该方法遵循 k-NN 密度估计的公式: $p(x|w_i) \approx \frac{k}{N_i V_i(x)}$, 其中 $V_i(x)$ 是包含 x 的 k 个 w_i 类邻居的最小球体体积。

在二维空间中, $V_i(x) = \pi d_{i,k}^2$, 其中 $d_{i,k}$ 是 x 到其第 k 个 w_i 类邻居的欧氏距离。为实现此计算, 代码首先使用 `manual_pairwise_sqdist` 计算测试点 `X_test` 到某类别 c 的所有训练点 `X_train_c` 的距离。

然后我们再对距离进行排序 (`dists.sort(axis=1)`), 并提取出第 k 个邻居的距离 `d_ik` (索引为 `self.k - 1`)。

最后, 在对数空间中计算类条件概率 (似然): $\log p(x|w_i) = \log(k) - \log(N_i) - \log(V_i)$, 其中 $\log(V_i) = \log(\pi) + 2\log(d_{i,k})$ 。获得对数似然 $p(x|w_i)$ 后, 后续的 LRT 与 MAP 决策流程与前面的实验完全一致。

Listing 11: k-NN 密度估计与决策

```

1 def predict_log_proba(self, X_test, priors, rule_type='MAP'):
2     log_probs = []
3
4     for c in self.classes_:
5         X_train_c = self.X_train_[c]
6         N_i = self.N_i_[c]
7
8         if N_i == 0 or self.k > N_i:
9             log_likelihood = np.full(X_test.shape[0], -np.inf)
10        else:
11            # 1. 计算 X_test 到该类所有训练点的平方距离
12            sq_dists = manual_pairwise_sqdist(X_test, X_train_c)
13            # (M, N_i)

```

```

13
14         # 2. 转换成真实距离
15         dists = np.sqrt(sq_dists)
16
17         # 3. 沿行排序 (对每个测试点), 找到 k 个最近的邻居
18         dists.sort(axis=1)
19
20         # 4. 找出第 k 个邻居的距离 (索引是 k-1)
21         d_ik = dists[:, self.k - 1] # (M,)
22
23         # 5. 处理 d_ik == 0 的情况 (测试点和训练点重合)
24         d_ik[d_ik == 0] = 1e-9 # 避免 log(0)
25
26         # 6. 计算  $\log(p(x|w_i)) = \log(k) - \log(N_i) - \log(V_i)$ 
27         log_Vi = np.log(np.pi) + 2 * np.log(d_ik)
28         log_likelihood = np.log(self.k) - np.log(N_i) -
29             log_Vi
30
31         # 7. 应用决策规则 (LRT / MAP)
32         if rule_type == 'MAP':
33             # ... (MAP 规则实现, 同中级实验节) ...
34             prior_prob = priors[int(c)-1]
35             if prior_prob == 0: log_prior = -np.inf
36             else: log_prior = np.log(prior_prob)
37             log_posterior = log_likelihood + log_prior
38             log_probs.append(log_posterior)
39         else: # LRT
40             log_probs.append(log_likelihood)
41
42     return np.stack(log_probs, axis=1)
43
44 def predict(self, X_test, priors, rule_type='MAP'):
45     log_posteriors = self.predict_log_proba(X_test, priors, rule_type)
46     pred_indices = np.argmax(log_posteriors, axis=1)
47     return self.classes_[pred_indices]

```

3. 实验执行

实验的执行部分通过一个循环实现, 该循环遍历实验要求的 k 值列表 `k_range = [1, 3, 5, 8, 10]`。在循环中, 代码实例化一个具有特定 k 值的 `KNNDensityClassifier`, 在全体数据上进行拟合与预测, 并记录 LRT 和 MAP 规则下的准确率。

Listing 12: k-NN 实验执行循环

```

1     k_range = [1, 3, 5, 8, 10]
2     results_knn = {}
3
4     fig, axes = plt.subplots(1, 3, figsize=(21, 6), sharey=True)
5     # ... (省略 fig.suptitle) ...

```

```

6
7     for ax, (name, data) in zip(axes, datasets.items()):
8         X, y = data['X'], data['y']
9         priors = data['config']['priors']
10
11         accs_map = []
12         accs_lrt = []
13         table_row_map = {}
14
15         for k in k_range:
16             knn_clf = KNNDensityClassifier(k=k)
17             knn_clf.fit(X, y)
18
19             # 评估 MAP
20             y_pred_map = knn_clf.predict(X, priors, rule_type='
                MAP')
21             acc_map = manual_accuracy(y, y_pred_map)
22             accs_map.append(acc_map)
23             table_row_map[k] = acc_map
24
25             # 评估 LRT
26             y_pred_lrt = knn_clf.predict(X, priors, rule_type='
                LRT')
27             # ... (LRT 评估代码, 此处省略) ...
28
29         results_knn[name] = {'MAP': table_row_map, 'LRT': ...}
30
31         # —— 绘图 ——
32         ax.plot(k_range, accs_lrt, 'o-', label='似然率测试(LRT)')
33         ax.plot(k_range, accs_map, 'o-', label='MAP规则')
34         # ... (后续绘图代码, 此处省略) ...

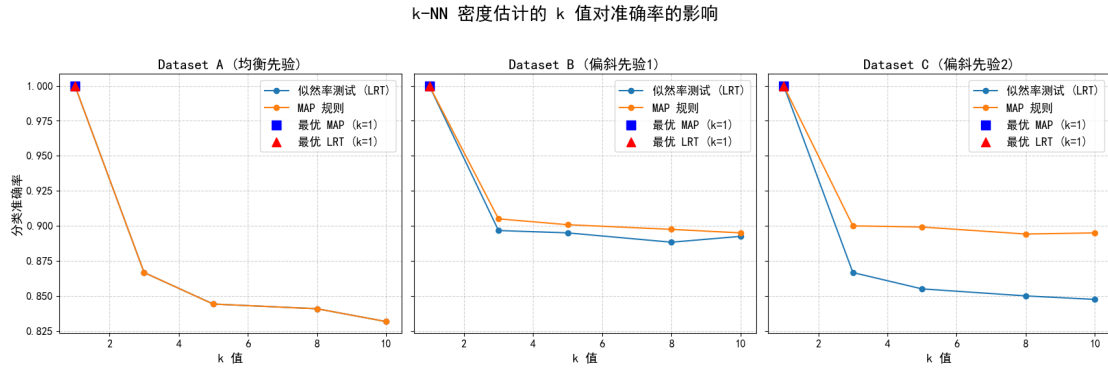
```

(三) 高级实验结果可视化

接下来我们根据实验结果来分析 k 值对 k-NN 密度估计分类器性能的影响。表 4 详细记录了不同 k 值下的分类准确率，图 9 则将其进行了可视化。

表 4: k-近邻方法中不同 k 值的分类准确率比较

数据集	分类规则	k=1	k=3	k=5	k=8	k=10
A	似然率测试	1.0000	0.8667	0.8442	0.8408	0.8317
	MAP 规则	1.0000	0.8667	0.8442	0.8408	0.8317
B	似然率测试	1.0000	0.8967	0.8950	0.8883	0.8925
	MAP 规则	1.0000	0.9050	0.9008	0.8975	0.8950
C	似然率测试	1.0000	0.8667	0.8550	0.8500	0.8475
	MAP 规则	1.0000	0.9000	0.8992	0.8942	0.8950

图 9: k-NN 密度估计的 k 值对分类准确率的影响

1. 高级实验结果分析

表 4 与图 9 共同揭示了 k 值对 k-NN 密度估计分类器性能的复杂影响。

一个最显著的现象是，在所有数据集中，当 $k = 1$ 时，LRT 与 MAP 规则均获得了 1.0 的完美准确率。这并非代表 $k = 1$ 是最优化模型，而是由于本实验在训练集上进行评估时产生的“过拟合”现象。当 $k = 1$ 时，k-NN 密度估计公式为 $p(x|w_i) \approx 1/(N_i \pi d_{i,1}^2)$ 。对于任意训练样本 x ，它总能找到其自身作为距离为零 ($d_{i,1} = 0$) 的最近邻，导致其自身类别的估计密度趋近于无穷大，从而被 100% 正确分类。这是一种极端过拟合的表现，缺乏泛化能力。

当 k 值从 1 增加到 3 时，准确率出现了急剧下降，这标志着模型从过拟合状态向更平滑、更具泛化能力的模型转变。随着 k 值继续增大 ($k = 3, 5, 8, 10$)，密度估计的平滑程度随之增加 (因为需要包含更多邻居，导致体积 $V_i(x)$ 增大)，分类准确率曲线也趋于稳定或缓慢变化。

对比两种决策规则，实验结果再次印证了先验概率的重要性。在数据集 A (均衡先验) 中，LRT 与 MAP 的准确率曲线 (如图 9 左图所示) 与数据 (如表 4 所示) 完全一致。而在数据集 B 和 C (偏斜先验) 中，对于所有 $k > 1$ 的情况，MAP 规则的性能 (橙色线) 均显著且稳定地优于 LRT 规则 (蓝色线)。

(四) 高级实验总结

高级实验成功实现了 k-NN 概率密度估计方法，并系统分析了参数 k 对分类性能的影响。实验结果表明， k 值的选择是 k-NN 密度估计的核心， k 值过小 (如 $k = 1$) 会导致模型在训练集上严重过拟合，而 k 值增大则会使密度估计趋于平滑。

与中级实验的 KDE 方法相似，k-NN 作为另一种非参数估计技术，同样证明了在先验概率不均衡时，结合 MAP 决策规则能取得远优于 LRT 规则的分类效果。

至此，我们完成了本次实验的所有实验要求！

七、 实验总结与讨论

(一) 实验中每种方法最好性能对比可视化

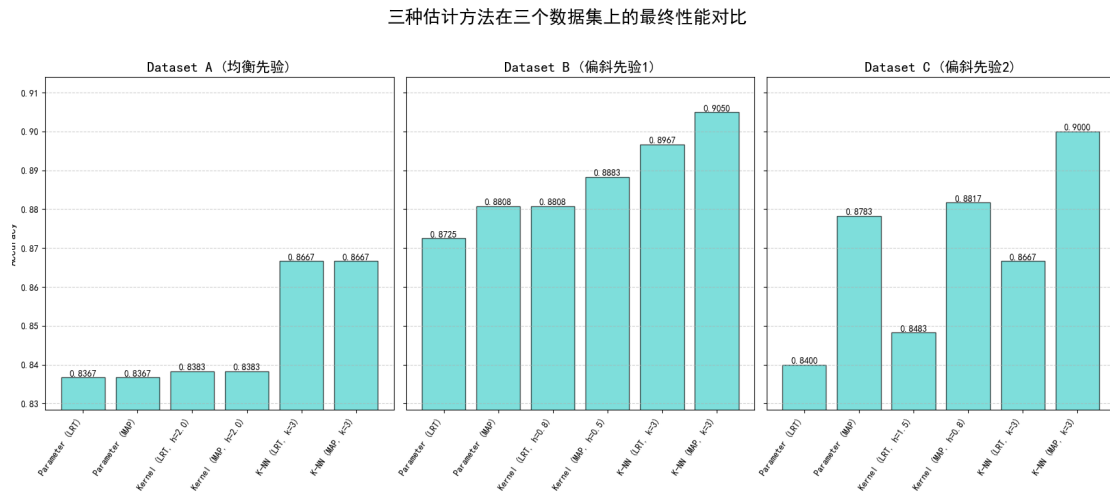


图 10: 三种估计方法在三个数据集上的最终性能对比。该图汇总了初级、中级、高级实验中各方法的最优结果（KDE 采用 CV 最优 h ，k-NN 采用 $k = 3$ 的结果）。

(二) 主要发现

本次实验通过三个递进的子实验（参数估计、KDE、k-NN）和三个不同先验的数据集（A、B、C），系统地验证了贝叶斯决策理论中的核心概念。主要发现总结如下：

- 先验概率的显著影响：**先验概率 $P(w_i)$ 对分类器性能具有决定性影响。在数据集 A（均衡先验）中，LRT 与 MAP 规则的性能几乎完全一致（如图 10 左图所示）。然而，在数据集 B 和 C（偏斜先验）中，MAP 规则在所有三种密度估计方法（参数、KDE、k-NN）上的表现均显著优于 LRT 规则。这有力地证明了，当类别分布不均衡时，LRT 规则（忽略先验）会导致次优决策，而 MAP 规则通过融合先验知识，能显著提高分类的总体准确率。
- 参数方法 vs 非参数方法：**
 - 参数方法：**由于本实验的数据（高斯分布）严格符合参数估计的模型假设，参数方法（初级实验）提供了强大的基准性能。其优点在于模型简洁、计算高效，并且在模型假设正确时，其结果（如数据集 C 中 MAP 的 0.8783）接近理论上的贝叶斯最优分类器。
 - 非参数方法：**KDE 与 k-NN 作为非参数方法，在不预知数据真实分布的情况下，均成功地从样本中拟合了多峰值的概率分布。如图 10 所示（B 和 C 图），优化的非参数方法（特别是 k-NN MAP）所达到的准确率（B: 0.9050, C: 0.9000）甚至超越了参数估计的基准。这可能是因为非参数方法的高度灵活性，使其在训练集评估时能更好地拟合样本数据的特定形态，但也反映出其对参数选择的敏感性。
- 参数选择的敏感性：**非参数方法的性能高度依赖于其平滑参数的选择，这就涉及到我们在各个模型中的偏差-方差权衡。

- **核密度估计中：**带宽 h 的选择至关重要。如中级实验图 5 所示， h 过小（如 0.1）导致模型过拟合，准确率低； h 过大（如 2.0）导致模型欠拟合（过度平滑），准确率亦低。5 折交叉验证是寻找最优 h （如 0.5 至 2.0）的必要手段。
- **k-近邻方法中：** k 值的选择同样关键。高级实验图 9 显示， $k = 1$ 时模型在训练集上出现 100% 准确率的极端过拟合现象。随着 k 增大到 3，模型性能才回归到一现实水平并逐渐稳定。这表明 k 值直接控制了密度估计的平滑程度，过小的 k 值对噪声极为敏感。

（三） 方法优缺点讨论

参数估计方法：

- **优点：**模型简单，所需存储空间小（仅需存储参数 μ 和 Σ ）；分类决策速度快；当模型假设（如本此次实验中的高斯分布）与数据真实分布一致时，估计结果非常准确且稳定。
- **缺点：**模型适应性差（高偏差）。如果数据不符合预设的模型，参数方法可能会产生严重错误的估计结果。

核密度估计方法（KDE）：

- **优点：**模型灵活（低偏差），无需对数据分布做任何假设，理论上可以拟合任意形状的密度函数；估计出的密度函数是连续且平滑的。
- **缺点：**计算与存储开销大（需要存储所有训练样本）；对带宽参数 h 的选择极为敏感；在高维空间中表现不佳。

k-近邻密度估计（k-NN）：

- **优点：**概念和实现相对简单；模型具有高度的局部自适应性，能很好地适应数据密度变化剧烈的区域。
- **缺点：**计算与存储开销大（需要存储所有样本并计算距离）；估计出的密度函数是不连续的（在第 k 个邻居处有跳变）；对参数 k 的选择极为敏感。

（四） 心得体会

通过本次实验，我从理论到实践，完整地实现了参数估计与非参数估计两种概率密度估计方法，并将其应用于贝叶斯分类决策中。

我最大的收获是直观地理解了贝叶斯定理在分类决策中的实际意义。初级实验中，数据集 B 和 C 的 LRT 与 MAP 规则的显著差异，使我深刻体会到先验概率 $P(w_i)$ 并非一个抽象的数学符号，而是一个决定分类边界、影响最终准确率的关键信息。

在完成中级和高级实验的过程中，我体会到了非参数方法的灵活性，以及这种灵活性所带来的代价——对平滑参数（ h 和 k ）的高度敏感性。5 折交叉验证和 k 值测试让我对模型选择中的偏差-方差权衡有了更具体的认识。

项目的完整实验报告与源代码已同步至 GitHub, 详情请见: <https://github.com/fan-tuaner614/Machine-Learning>